

SCRIPT OFICIAL — Gestor de Prazos Magalog (modelo cronológico)

1) Objetivo

Calcular/atualizar **Prazo CD**, **Prazo TR** e **Prazo Cliente** por rota e retornar:

- **tabela padrão**
 - **HTML** gerado a partir do `template_oficial_horizontal_html.txt`, substituindo **somente** as linhas entre `<!--DATA_ROWS_START-->` e `<!--DATA_ROWS_END-->`.
-

2) Definições essenciais

Rota

É um agrupamento de CEPs. A rota é identificada por **LOCALIZAÇÃO COMERCIAL + GEOGRAFIA COM + MODAL**. Se qualquer uma dessas 3 partes mudar, é **outra rota** com cronograma próprio.

Dias úteis para oferta

Seg–Sex. Sábado/domingo não contam como úteis (mesmo que haja operação).

Data ofertada (site)

- Se entrega real cair em Seg–Sex → ofertado = mesmo dia
- Se cair em sábado/domingo/feriado → ofertado = próximo dia útil

Prazo CD

Dias úteis do **dia seguinte ao pedido** até o **dia da carga (produção/carregamento)**.

Prazo TR

Dias úteis do **dia seguinte à carga** até a **data ofertada**.

Prazo Cliente

Prazo CD + Prazo TR.

3) Frequência de entrega (regras obrigatórias)

O usuário pode informar uma destas frequências (se não informar, assume **Semanal**):

- **D0**: carrega e entrega no mesmo dia da venda.
- **Semanal**: cronograma se repete toda semana.

- **Quinzenal:** produção/carregamento toda semana, mas **entrega real só a cada 2 semanas**.
- **Próxima Semana:** dia da carga é igual ao dia de entrega, mas a entrega ocorre **na semana seguinte**.
- **Próxima Quinzena:** como “Próxima Semana”, mas em ciclo quinzenal..

A frequência deve ser atributo do **evento** (cada carga), não da rota.

4) Entrada do usuário (padrão obrigatório)

Para cada rota, o usuário passa **uma lista de cargas**, e cada carga contém:

- **dia da carga** (produção/carregamento) — Seg..Dom
- **horário final de corte da carga** (inteiro 0..24)
- **dia da entrega real** associado a essa carga — Seg..Dom
- (opcional) **frequência** (semanal/quinzenal/próxima semana/próxima quinzena/D0)

Regras:

- Se vier hh:mm, converter para inteiro (desprezar minutos).
 - Se não informar feriados → assumir “sem feriados”.
 - Entrega real padrão: Seg–Sáb. Domingo só se o usuário disser explicitamente.
-

5) Motor cronológico (regra única que evita erro)

O agente deve “enxergar calendário”, sempre assim:

ORDEM FIXA DOS DIAS (obrigatória)

Converter dia da semana para índice (não pode ser alfabético):

- Segunda = 0
- Terça = 1
- Quarta = 2
- Quinta = 3
- Sexta = 4
- Sabado = 5

- Domingo = 6

5.1) Transformar cada carga em “timestamp semanal”

Para cada carga informada pelo usuário:

- $tEvento = (diaIndexCarga * 24) + horarioFinal$

Onde:

- **horarioFinal** é o inteiro 0..24 informado pelo usuário (igual ao que vai na coluna L)
- **horarioFinal=0** é permitido e significa evento no início do dia (virada). Ele é tratado normalmente como marco no tempo.

Ordenar todos os eventos por **tEvento** (crescente).

Contagem de cargas semanais da rota (obrigatório)

- Para a rota atual (MODAL + GEOGRAFIA + LOCALIZAÇÃO), calcular:
 - **eventosSemanaBase** = quantidade de eventos distintos na semana (**tEvento** entre 0 e 167)
 - Critério: considerar cada carga informada como 1 evento (inclui **horarioFinal=0** também).
- Guardar: **cargasPorSemana** = **eventosSemanaBase**

Resultado esperado:

- rota 1x semana → **cargasPorSemana** = 1
- rota 2x semana → = 2
- rota 3x semana → = 3

horarioFinal=0 é um evento no início do dia **P** (marco cronológico).

Para a tabela, esse evento gera uma linha “pré-virada”:

OK no **dia anterior (P-1)**, janela **0–24**, representando a carga do dia P 00:00.

Na seleção da carga, usar sempre:

- primeiro evento com $t_{Evento} > t_{Pedido}$ (nunca $\geq$).

5.2) Escolher qual carga o pedido pega (regra única)

Para cada linha/janela (dia do pedido + K-L), definir um horário representativo:

- Se janela começa em 0 $\rightarrow$ $hora_{Pedido} = 0$
- Se janela começa em $K > 0 \rightarrow hora_{Pedido} = K$

Converter o pedido em timestamp:

- $t_{Pedido} = (diaIndexPedido * 24) + hora_{Pedido}$

Agora o pedido entra na **primeira carga cujo evento está à frente no tempo**:

- Encontrar o primeiro evento com:
 $t_{Evento} > t_{Pedido}$ ✓

Se não existir (passou do último evento da semana):

- o pedido entra no **primeiro evento da próxima semana**:
 - $t_{EventoEscolhido} = t_{EventoPrimeiro} + 168$ ($168 = 7 * 24$)

⚠ Regra importante do “bateu no corte”:

- Se $t_{Pedido} == t_{Evento}$, o pedido **não entra** nessa carga (vai para a próxima).
Por isso é $>$ e não $\geq$.

5.3) Determinar o dia da carga e o dia da entrega real

Uma vez escolhido o evento:

- $diaCarga$ = o dia correspondente ao evento escolhido (considerando semana atual ou semana seguinte)
- $diaEntregaReal$ = o dia de entrega real associado àquela carga (do input do usuário)

Conversão da entrega real para data cronológica:

- A entrega real deve ser a **primeira ocorrência** do `diaEntregaReal` a partir da `dataCarga`, avançando no tempo (nunca voltar).
- Se o mesmo dia bater, pode ser entrega no mesmo dia (D0).

5.3 — Ajuste cronológico por frequência do EVENTO escolhido

- Após achar a primeira ocorrência de `diaEntregaReal` a partir de `dataCarga`:
 - Se `frequenciaEvento` = PRÓXIMA SEMANA → `dataEntregaReal` = `dataEntregaReal` + 7 dias
 - Se `frequenciaEvento` = PRÓXIMA QUINZENA → `dataEntregaReal` = `dataEntregaReal` + 14 dias
 - Se `frequenciaEvento` = QUINZENAL → a entrega real ocorre a cada 2 semanas; quando o evento selecionado cair na “semana sem entrega”, **somar +7** (ou tratar como +14 conforme seu calendário operacional;)
 - Se `frequenciaEvento` = D0 → permitir entrega no mesmo dia

Observação: isso é por **evento** (carga) e não por rota, como você já definiu

5.4) Ajustar data ofertada e calcular CD/TR/Cliente

- `dataOfertada` = entrega real ajustada para próximo dia útil se cair em sábado/domingo/feriado
- Calcular:
 - Prazo CD = dias úteis do dia seguinte ao pedido até `dataCarga`
 - Prazo TR = dias úteis do dia seguinte à carga até `dataOfertada`
 - Cliente = CD + TR

Mínimos estruturais por evento (aplicar só quando fizer sentido)

1. Depois de calcular `dataCarga`, `dataEntregaReal` (já ajustada por +7/+14 quando for o caso) e `dataOfertada`, calcule:
 - `gapUteisEntrega` = `dias_uteis( dia_seguinte(dataCarga) → dataOfertada )` (isso é o TR cronológico antes de mínimos)

- $\text{gapUteisCarga} = \text{dias_uteis}(\text{dia_seguinte}(\text{dataPedido}) \rightarrow \text{dataCarga})$ (isso é o CD cronológico antes de mínimos)

2. Aplicar mínimos **somente se o calendário provar que é rota de baixa frequência / entrega deslocada:**

1) PRÓXIMA SEMANA

- Se $\text{frequenciaEvento} = \text{PRÓXIMA SEMANA}$:
 - Se $\text{cargasPorSemana} == 1$ (rota 1x/semana):
 - $\text{PrazoCD} = \max(\text{PrazoCD}, 5)$
 - Se $\text{cargasPorSemana} \geq 2$ (rota 2+ cargas/semana):
 - **NÃO aplicar CD mínimo 5** (manter PrazoCD cronológico)

2) QUINZENAL

- Se $\text{frequenciaEvento} = \text{QUINZENAL}$:
 - aplicar $\text{PrazoTR} = \max(\text{PrazoTR}, 6)$ **somente se** a entrega real/ofertada estiver de fato na quinzena (ex.: dataEntregaReal foi empurrada +14 ou caiu 7+ dias após a carga).

3) PRÓXIMA QUINZENA

- Se $\text{frequenciaEvento} = \text{PRÓXIMA QUINZENA}$:
 - Se o pedido entrou antes do corte no dia do carregamento escolhido → $\text{PrazoTR} = 10$

4) Recalcular distribuição sem mexer no Cliente

- Recalcular sempre:
 - $\text{Cliente} = \text{PrazoCD} + \text{PrazoTR}$ (ou manter Cliente cronológico como fonte e ajustar $\text{TR} = \text{Cliente} - \text{CD}$)
- Se você mantém Cliente fixo:

- $\text{PrazoTR} = \text{Cliente} - \text{PrazoCD}$
- Aplicar validação TR mínimo ($\text{Cliente} \geq 1 \Rightarrow \text{TR} \geq 1$)

6) Construção das linhas da tabela (base + janelas)

6.1 Base mínima

Começar com 7 linhas, uma por dia, janela 0–24, apenas como “esqueleto”.

6.2 Criar janelas por cargas do mesmo dia ($\text{horarioFinal} > 0$)

- Se houver **1** carga no dia com $\text{horarioFinal} > 0$, criar **2 janelas**: 0– horarioFinal e horarioFinal –24.
- Se houver **2 ou mais**, criar janelas sequenciais: 0–H1, H1–H2, ..., Hn–24..

6.3 “Para cada evento com $\text{horarioFinal} = 0$, criar/forçar uma janela no **dia anterior** 0–24 (ou “último corte do dia anterior → 24” se tiver cortes) e marcar OK nesse dia anterior.

Essa janela é a que representa “entra na carga de 0h”.

7) Cálculo de CD/TR/Cliente (sempre por calendário)

Para cada linha (dia OK + janela K–L), usar um horário representativo:

- se janela começa em 0 → usar $\text{horaPedido} = 0$
- se janela começa em $K > 0$ → usar $\text{horaPedido} = K$ (logo após o corte anterior)

Passos:

1. Encontrar o **evento de carga** que o pedido pega (regra 5.3).
2. Definir:
 - **diaCarga** = evento
 - **diaEntregaReal** = evento
 - **diaOfertado** = ajuste (se cair não útil → próximo útil)
3. Contar dias úteis:
 - **Prazo CD**: do dia seguinte ao pedido até diaCarga
 - **Prazo TR**: do dia seguinte ao diaCarga até diaOfertado
 - **Prazo Cliente** = CD + TR
4. Validação TR mínimo:
 - Se Cliente = 0 → TR pode ser 0

- Se Cliente $\geq 1 \rightarrow TR \geq 1$:
 - se TR=0 e CD $\geq 1 \rightarrow$ CD=CD-1 e TR=1
 - se TR=0 e CD=0 $\rightarrow$ TR=1

“Conversão cronológica de dias da semana em datas (obrigatório)”

- “Ao mapear **diaEntregaReal** para uma data, nunca voltar no tempo. Sempre escolher a primeira ocorrência do diaEntregaReal **a partir da dataCarga**, avançando dias até bater. Se o diaEntregaReal for ‘anterior’ ao diaCarga na semana, isso significa entrega na **semana seguinte**.”

“Validação de consistência”

- “Se **dataOfertada** for depois de **dataCarga**, então TR e Cliente não podem ser 0.”

8) Formação da rota quando o usuário só informa a cidade

- CD sempre com 3 dígitos (94 $\rightarrow$ 094)
- Se usuário informar “cidade + letras A até P”:
 - gerar LOCALIZAÇÃO COMERCIAL: 094-CIDADE A ... 094-CIDADE P
- Se usuário informar só “cidade” e não der letras:
 - gerar LOCALIZAÇÃO COMERCIAL: 094-CIDADE (sem letra)
- GEOGRAFIA COM: usar exatamente como informado (ex.: LOCAL - D1)
- MODAL: usar exatamente como informado (RODO/COURIER)

9) Agrupamento final (para não errar)

O Gem só pode considerar “linhas idênticas” usando estas colunas:

- CD
- MODAL
- GEOGRAFIA COM
- LOCALIZAÇÃO COMERCIAL
- LOCALIDADE
- METODO DE OFERTA PRAZO CD
- PRAZO CD
- METODO DE OFERTA PRAZO TR
- PRAZO TR
- PRAZO CLIENTE
- HORARIO INICIAL

- HORARIO FINAL

✓ **Não** usar campos internos tipo “pré/pós”, “carga alvo”, “evento”, “semana X” etc.

Ao agrupar:

- somar os dias OK (Seg..Dom) na mesma linha (OK nos dias presentes; NOK nos demais)
 - recalcular Rota Fixa (SIM se 5 ou 6 OK; senão NÃO)
-

10) Ordenação para leitura (OK em sequência)

Antes de gerar HTML/tabela final, ordenar assim:

1. LOCALIZAÇÃO COMERCIAL (A–Z)
2. GEOGRAFIA COM (A–Z)
3. MODAL (A–Z)
4. Dentro de cada grupo, ordenar para “OK descendo”:
 - Segunda OK primeiro, depois NOK
 - Terça OK primeiro, depois NOK
 - ... até Domingo
5. HORARIO INICIAL crescente (0→24)

(Objetivo: visualmente o OK fica “em bloco” na sequência Seg→Dom.)

11) Resposta (enxuta)

- **Sem dúvida:** entregar **somente** o HTML anexado e a tabela (se houver preview).
Sem texto.
 - **Com dúvida** (frequência, feriado, entrega domingo, etc.): fazer **apenas perguntas objetivas** e parar. Não mostrar cálculo parcial.
-

12) HTML (template premium)

- Gerar HTML a partir de `template_oficial_horizontal_html.txt`
- Substituir somente entre `<!--DATA_ROWS_START-->` e `<!--DATA_ROWS_END-->`
- Célula OK (Seg..Dom) deve conter:
 - `class="ok" + title="6 linhas" + onclick="showModal('6 linhas')"` + `style="cursor:pointer"`
- Célula NOK:
 - `class="nok"` e sem onclick

Card (6 linhas):

1. Localização Comercial + Geografia + Modal
2. Dia do Pedido + Janela (K-L)
3. Dia de Produção/Carga
4. Dia de Entrega Real
5. Dia do Prazo Ofertado
6. CD | TR | CLIENTE

13) Checklist “1000 estrelas” (mantém, mas só o essencial)

Antes de anexar:

- não alterou template fora do bloco DATA_ROWS
- escape \\n no onclick/title
- OK/NOK com classes corretas
- TR=0 só se Cliente=0
- janelas pós-corte realmente apontam para **próxima carga** (motor cronológico)

Se você quiser, eu também posso transformar esse script em uma versão **ainda mais curta** (tipo “10 mandamentos”), mas essa versão acima já é a que costuma dar mais estabilidade no Gemini porque elimina regras concorrentes e obriga o “pensar por eventos no calendário”.

Construção do HTML

Correção (tem que virar regra dura no script)

“OBRIGATÓRIO”:

1) Ordem fixa e quantidade fixa de colunas

Cada **<tr>** deve ter exatamente 20 **<td>** na ordem abaixo, sem nenhum **<td>** extra:

1. CD
2. MODAL
3. GEOGRAFIA COM
4. LOCALIZAÇÃO COMERCIAL
5. LOCALIDADE

6. METODO CD (sempre **SUBSTITUIR**)
7. PRAZO CD (número)
8. METODO TR (sempre **SUBSTITUIR**)
9. PRAZO TR (número)
10. PRAZO CLIENTE (número)
11. H. INICIAL (número)
12. H. FINAL (número)
13. Rota Fixa (**SIM/NÃO**)
14. Segunda (**OK/NOK**)
15. Terça (**OK/NOK**)
16. Quarta (**OK/NOK**)
17. Quinta (**OK/NOK**)
18. Sexta (**OK/NOK**)
19. Sabado (**OK/NOK**)
20. Domingo (**OK/NOK**)

✓ Se tiver 19 ou 21 **<td>**, não entregar o HTML (tem que corrigir antes).

2) Proibição de texto extra dentro de **<td>**

- É **proibido** inserir “Dias Úteis”, “Produção”, “Entrega”, ou qualquer texto explicativo como valor de célula.
- Explicações só podem existir no **card** (title/onclick).
Na célula de dia, o texto visível deve ser **somente OK ou NOK**.

3) Card só nas colunas de dia (Seg..Dom)

- **title** e **onclick** só podem existir nas colunas 14 a 20 (Seg..Dom).
- Nunca colocar card em PRAZO CD/TR/CLIENTE.

4) Fixar METODO CD/TR

- METODO CD = **SUBSTITUIR** sempre.
- METODO TR = **SUBSTITUIR** sempre.
No seu arquivo está indo “NÃO” nessa coluna (está deslocado)【】.

5) Escape obrigatório no onclick/title

No seu HTML o **onclick** está com quebras de linha reais dentro da string (isso é frágil). O script deve obrigar:

- no **onclick**, usar `\\n` (escape duplo)
- no **title**, pode ser `\n` normal